

P2065 [TJOI2011] 卡片 深度学习笔记

aaakkk

December 20, 2025

1 题目描述

桌子上有 m 张蓝色卡片与 n 张红色卡片，每张卡片上有一个大于 1 的整数。你每次可以拿走一对卡片（一张蓝色和一张红色），前提是它们上面数字的最大公约数（GCD）大于 1。问：通过最优策略，最多可以拿走多少组卡片？

数据范围： $T \leq 100$, $m, n \leq 500$, 卡片上的数字 $v \in (1, 10^7]$ 。

2 关键思路与知识：质因子中转模型

本题的本质是二分图最大匹配。

若直接在满足 $\gcd(b_i, r_j) > 1$ 的蓝红卡片间连边，最坏情况下边数可达 $500 \times 500 = 2.5 \times 10^5$ 。在多组数据下，由于边数过多且 GCD 运算频繁，直接建图会导致严重超时（TLE）。

关键知识： 任何两个大于 1 且 GCD 超过 1 的整数，必然共同拥有至少一个质因子。因此，我们可以通过引入“质因子节点”作为中转站来重构图模型：

- **流量路径设计：** 源点 $S \rightarrow$ 蓝色卡片 $\rightarrow$ 该蓝卡的所有质因子节点 $\rightarrow$ 拥有相同质因子的红色卡片 $\rightarrow$ 汇点 T 。
- **边数大幅压缩：** 由于 10^7 内的整数最多只有 7 个不同质因子，这种方式将边数从 $O(M \cdot N)$ 降低到了 $O((M + N) \cdot \omega(V))$ 。

3 关键错误详细梳理与 AC 对比

3.1 错误 1：朴素 GCD 连边（导致 TLE）

错误做法： 使用双重循环对每一对 (b_i, r_j) 进行判断并连边。

AC 对比： AC 代码放弃了卡片间的直接连边，改为通过质因子建立联系。这不仅省去了海量的 GCD 运算，还将网络流中的边数压缩了几个数量级，确保在 100 组数据内通过。

3.2 错误 2：网络流向逻辑混乱（导致死循环）

错误做法： 误将蓝卡和红卡都连向质因子节点（即 $Blue \rightarrow Prime$ 且 $Red \rightarrow Prime$ ）。

AC 对比： 在网络流中，路径必须是从 S 到 T 的单一向。AC 代码修正了方向：蓝色卡片连接到质因子，质因子再连接到红色卡片。这样流量才能顺着“蓝 $\rightarrow$ 质 $\rightarrow$ 红”的路径通过增广路到达汇点。

3.3 错误 3：质数分解效率极低（导致超时）

错误做法： 使用 ‘for(int v : p)’ 线性遍历巨大的素数表来分解每个卡片的数字。

AC 对比： AC 代码采用了基于**试除法**的优化：循环只需要运行到 $\sqrt{val}$ ，剩下的 ‘temp > 1’ 部分即为最后一个大质因子。这种方式将单次分解的复杂度从 $O(\text{素数个数})$ 降到了 $O(\sqrt{V})$ ，极大地减少了建图时间。

3.4 错误 4：初始化范围不足（导致结果污染）

错误做法： 仅重置了前 $n + m$ 个节点的 'head' 数组。

AC 对比： 由于质因子节点的编号是动态增加的（通常在 $n + m$ 之后），如果不清理到最新的汇点 t ，上一组数据的残余边信息会保留在图中，导致结果完全错误或数组越界。AC 代码通过 'for(int i = 0; i <= t; ++i) head[i] = 0;' 确保了环境的彻底清空。

4 AC 代码实现

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4
5  // 1~ mm+1~m+ n
6  constexpr int maxm = 2e6 + 10, maxn = 2e5 + 10;
7
8  vector<int> p;
9  bool is[(int)2e7 + 10];
10
11 //
12 void ouler_seive() {
13     is[1] = 1;
14     for(int i = 2; i <= 2e7; ++i) {
15         if(!is[i]) p.emplace_back(i);
16         for(int j = 0; j < p.size() && p[j] * i <= 2e7; ++j) {
17             is[i * p[j]] = 1;
18             if(i % p[j] == 0) break;
19         }
20     }
21 }
22
23 struct edge {
24     int to, w, nxt;
25 } edges[maxm];
26
27 int head[maxn], enums = 1, levels[maxn], cur[maxn], n, m, s, t, ans = 0;
28
29 void add_edge(int x, int y, int w) {
30     edges[++enums] = {y, w, head[x]}; head[x] = enums;
31     edges[++enums] = {x, 0, head[y]}; head[y] = enums;
32 }
33
34 bool bfs() {
35     for(int i = 0; i <= t; ++i) { cur[i] = head[i]; levels[i] = 0; }
36     queue<int> q; q.emplace(s);
37     levels[s] = 1;
38     while(!q.empty()) {
39         int u = q.front(); q.pop();
40         for(int i = head[u]; i; i = edges[i].nxt) {
41             int v = edges[i].to;
42             if(!levels[v] && edges[i].w) {
43                 levels[v] = levels[u] + 1;
44                 q.emplace(v);
45                 if(v == t) return true;
46             }
47         }
48     }
```

```

48     }
49     return false;
50 }
51
52 int dfs(int u, int limit) {
53     if(u == t || !limit) return limit;
54     int flow = 0;
55     for(int& i = cur[u]; i; i = edges[i].nxt) {
56         int v = edges[i].to;
57         if(levels[v] == levels[u] + 1 && edges[i].w) {
58             int f = dfs(v, min(limit - flow, edges[i].w));
59             if(f) {
60                 flow += f; edges[i].w -= f; edges[i ^ 1].w += f;
61                 if(flow == limit) return flow;
62             } else levels[v] = 0;
63         }
64     }
65     return flow;
66 }
67
68 signed main() {
69     ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
70     ouler_seive();
71     int tt; cin >> tt;
72     while(tt--) {
73         unordered_map<int, int> ys; //
74         ans = 0, enums = 1;
75         cin >> m >> n;
76         int id_counter = 0;
77
78         // ->
79         for(int i = 1; i <= m; ++i) {
80             int val; cin >> val;
81             for(int j = 0; p[j] * p[j] <= val; ++j) {
82                 if(val % p[j] == 0) {
83                     if(!ys.count(p[j])) ys[p[j]] = ++id_counter + n + m;
84                     add_edge(i, ys[p[j]], 1);
85                     while(val % p[j] == 0) val /= p[j];
86                 }
87             }
88             if(val > 1) {
89                 if(!ys.count(val)) ys[val] = ++id_counter + n + m;
90                 add_edge(i, ys[val], 1);
91             }
92         }
93
94         // ->
95         for(int i = 1; i <= n; ++i) {
96             int val; cin >> val;
97             for(int j = 0; p[j] * p[j] <= val; ++j) {
98                 if(val % p[j] == 0) {
99                     if(!ys.count(p[j])) ys[p[j]] = ++id_counter + n + m;
100                     add_edge(ys[p[j]], m + i, 1);
101                     while(val % p[j] == 0) val /= p[j];
102                 }
103             }
104             if(val > 1) {
105                 if(!ys.count(val)) ys[val] = ++id_counter + n + m;

```

```

106         add_edge(ys[val], m + i, 1);
107     }
108 }
109
110     s = 0, t = id_counter + n + m + 1;
111     for(int i = 1; i <= m; ++i) add_edge(s, i, 1);
112     for(int i = 1; i <= n; ++i) add_edge(m + i, t, 1);
113
114     while(bfs()) ans += dfs(s, 1e18);
115     cout << ans << "\n";
116
117     // head
118     for(int i = 0; i <= t; ++i) head[i] = 0;
119 }
120 return 0;
121 }

```

署名: aaakkk